Passageworks: System Evaluation and Load Characterization

Ayush Morbar | November 2026 | ayushmorbar/passageworks

An asynchronous, four-stage research assistant grounded strictly in three arXiv papers, evaluated under burst traffic, provider rate limits, and service failures. Benchmarked against live Groq LPUs (`qwen/qwen3.8-27b`) and an empirical stand-in model service.

1. What I built and why

Passageworks is an asynchronous question-answering pipeline grounded strictly in three arXiv papers: *Attention Is All You Need* (1706.03762v7), *Moshi* (2410.00037v1), and *Dynamic Chunking* (2507.07955v2). Its purpose is to investigate the behavior of the LLM pipeline under burst arrival schedules, provider quota, and service degradation.

The pipeline executes four discrete stages: `CLASSIFY` (a lightweight classifier model call routing out-of-scope queries before retrieval or drafting), `RETRIEVE` (in-memory BM25 chunk search), `DRAFT` (initial synthesis with explicit citations), and `VERIFY` (independent LLM audit of claim-passage alignment).

The single architectural decision that shaped the system most was **buffered verification**: withholding all output until an independent LLM validation step confirms exact claim-passage alignment against source citations `[Paper, Section, Page]`. The pipeline buffers output until verification passes. This prevents the release of unverified drafts, but does not guarantee factual correctness.

2. The real service and my stand-in

To parameterize the pipeline, I probed the live Groq API (`qwen/qwen3.8-27b`) on November 9, 2026, using HTTP response headers and empirical test calls.

Table 1: Limits I found in the real service

Limit	Value found	How I found it	Conf.
Req / min	30 RPM	Groq docs; 429 lockouts on > 30 RPM	Med
Tokens / min	7k–8k ITPM	Header `limit-tokens: 8000`; 429 payload cites `Limit 7000 ITPM`	High
Req / day	1,000 RPD	Header `limit-requests: 1000`	High
Tokens / day	500k TPD	Groq developer free-tier quota	High
In flight	∼ 3–5 conc.	Inferred provider ceiling; client semaphore set to 4	Med
On limit hit	HTTP 429	Header `retry-after` and JSON `rate_limit_exceeded`	High
Recovery	210ms–13.6s	Header `reset-tokens` (sub-second); 429 `try again in Xs`	Med

Stand-in Calibration: Calibrated from 5 controlled baseline probes against live Groq endpoints measuring TTFT (median 178.5 ms), streaming token cadence (∼ 65 tok/s via SSE), concurrency degradation, and exact HTTP response headers.

A Number We Could Not Have Guessed Without Running It: Groq's sub-second token replenishment slice interval: `x-ratelimit-reset-tokens` reported fractional credit reset times between 210 ms and 630 ms rather than whole seconds, while 429 payloads disclosed exact token balances down to single tokens (`Limit 7000, Used 6754, Requested 1834. Please try again in 13.611428571s`).

Load-Dependent Slowdown: Adds load-dependent latency to approximate observed degradation (each concurrent request beyond 4 adds 12% jitter); it does not reproduce the provider's internal hardware or scheduling behavior.

Differences from Real Service: The Stand-in emits deterministic text rather than executing full LLM inference, cannot simulate GPU memory bandwidth contention or hardware cache misses, and replenishes token buckets on discrete tick intervals rather than continuous streaming.

3. Results

Table 2a: Latency, one row per load scenario

Scenario	Src	Req	TTFT p50/p95 (ms)	Total p50/p95 (ms)
steady	stand-in	25	1566.5 / 1854.3	9763.2 / 20354.0
burst	stand-in	35	1664.1 / 1830.0	0.1 / 1699.1
sustained_ovl	stand-in	45	1681.3 / 2294.8	0.1 / 1686.9
halved_limit	stand-in	25	1676.5 / 1702.9	0.1 / 1673.7
daily_exhaust	stand-in	15	not measured	8.3 / 394.6
real_service	real	6	1723.1 / 2389.4	2093.2 / 5795.0

Accounting Note on Latency: TTFT = time to first user-visible token. First-token percentiles describe successful completions (`status == 'ok'`). Total-time percentiles are computed across all dispatched requests, accurately capturing instantaneous load-shedding rejections (0.1 ms) alongside queued and completed requests. For successful completions alone, burst Total Time p50 is 1,664.1 ms, sustained overload Total Time p50 is 1,681.3 ms, and halved limit is 1,676.5 ms. For `daily_budget_exhausted`, zero requests succeeded, so percentiles describe all 15 rejection latencies. For `real_service`, total-time percentiles describe all six dispatched requests, capturing provider 429 lockouts.

Table 2b: Outcomes, same rows

Scenario	Succ%	429%	Retr.	Tok/ans	Cost/1k
steady	32.0%	68.0%	1.36	1998.4	$0.105
burst	17.1%	82.9%	0.00	2148.7	$0.113
sustained_ovl	13.3%	86.7%	0.00	2148.7	$0.113
halved_limit	12.0%	88.0%	0.00	2473.3	$0.130
daily_exhaust	0.0%	100.0%	1.00	0.0	$0.000
real_service	50.0%	50.0%	0.67	2418.3	$0.138

Real-Service Quality Breakdown: No incomplete draft was released in the recorded real-service run. The three `wrong` outcomes were quota-related non-completions, not factual answer errors. Of the three real-service requests that were completed, two were correct (`real-groq-0001`, `0002`) and one was a correct refusal (`real-groq-0005`). The other three did not produce a complete answer due to quota limits (`real-groq-0003` and `0004` hit 429 ITPM exhaustion during revision/verification; `real-groq-0006` was intercepted at admission at 7,255/8,000 active TPM).

4. Where the time goes

Analysis of real Groq execution for `real-groq-0001` (*"What is the formula for Scaled Dot-Product Attention in the Transformer?"*):

- **Waiting time:** 0.0 ms (admission slot immediately available).
- **Working time:** 2,462.7 ms (2,463.4 ms total elapsed).
  - `CLASSIFY`: 1,092.0 ms (378 prompt tok, 62 completion tok).
  - `RETRIEVE`: 2.8 ms (0 network tok) — local BM25 top-3 chunks.

  - `DRAFT`: 583.3 ms (1,210 prompt tok, 102 completion tok).
  - `VERIFY`: 784.6 ms (1,301 prompt tok, 70 completion tok).
- **Retrying time:** 0.0 ms (zero retries required).
- **Total tokens:** 2,889 prompt, 234 completion (3,123 total).
- **Delay before first token:** All 2,463.4 ms is delay before the first token because output is buffered until verification passes.

5. When a limit is hit

What the system does at limits: Client sliding-window counters monitor active TPM and RPM. When rolling usage nears exhaustion ($< 1,500$ tokens remaining), retrieval shifts dynamically from 5 chunks to 3 chunks. If insufficient quota remains for a draft-verify cycle, newly arriving requests are shed at admission in < 0.1 ms. At the daily limit (500k tokens), subsequent requests are rejected locally until 00:00 UTC.

Who is delayed, dropped, or gets a weaker answer: In `steady`, requests queue behind the concurrency limiter (`max_concurrency=4`), waiting up to 20.4 s. In `burst`, requests wait until the queue hits cap (18), after which excess is shed (0.1 ms). Nobody receives a weaker answer: responses are either fully verified with citations `[Paper, Section, Page]`, or refused cleanly.

Retrieval: quality versus cost

Table 3: Retrieval settings

Setting	Q	Corr%	Wrng%	Ref%	Tok	TTFT p50/p95 (ms)
A: 250w, 5p, no	18	50.0%	5.6%	22.2%	3176.8	1631 / 3910
B: 250w, 3p, no	18	50.0%	5.6%	22.2%	2171.7	1599 / 3932
C: 250w, 5p, yes	18	50.0%	5.6%	22.2%	3162.4	1634 / 3908

Evaluation Disclosure: Table 3 measures pipeline mechanics using the stand-in. It does not establish real-model answer quality across the three settings. All 60 records are published in `results/eval_records.csv`.

- **Answerable (13 Q):** 9 answered correctly with citations (69.2%). 4 (30.8% of answerable; 22.2% of total 18) incorrectly refused (`refused_wrongly`) due to conservative classifier gates.
- **Unanswerable (5 Q):** 4 correctly refused (80.0%), 1 wrong (RLHF trap).
- **Token Savings:** Setting B uses 31.6% fewer tokens per attempt (2,171.7 vs. 3,176.8) while preserving the measured grade distribution. When the quota is low, the retrieval automatically goes back to Setting B.

How this library will mislead a naive retriever

1. **Section Title Ambiguity:** Moshi and Dynamic Chunking both share generic titles ("3 Architecture", "4 Experiments"). Naive BM25 blends passages without distinguishing Moshi audio from H-Net chunking. *Example:* "What architectural changes were made in Section 3 to support streaming?"
2. **Subword vs. Continuous Token Collisions:** "token representations" matches Attention's BPE subwords (37k vocabulary) and Moshi's continuous audio quantizers (Mimi RVQ at 12.5 Hz). *Example:* "How are continuous sequence tokens quantized in the representation layer?"
3. **SSM / Mamba-2 Entanglement:** Dynamic Chunking evaluates Mamba-2 backbones in H-Net; naive keyword filters either wrongly reject queries or retrieve external non-corpus literature. *Example:* "What state-space models are integrated into the hierarchical backbone?"

6. Retries

Under burst and sustained load, retrying requests that hit rate limits made degradation worse. In trace `real-groq-0003`, after consuming 3,347 tokens across `CLASSIFY` and `DRAFT`, `VERIFY` requested 1,834 tokens against an active usage of 6,754 tokens, returning HTTP 429: `Limit 7000, Used 6754, Requested 1834. Please try again in 13.611s`. After a 1.5s client backoff, the retry immediately hit the same 429 condition (`retry-after: 13.6s`), consuming 5,911.6 ms across 2 retries without completing. Retrying against a minute-level quota window without waiting for the full lockout period compounded latency and blocked worker slots. **Mitigation:** We set `max_retries=2` and added a fast-fail cutoff (`if retry_after > 10.0: abort`).

7. What the person sees

When daily token budget (500k tokens) is exhausted (simulated at 3:00 PM), arriving requests are halted. The person sees:

```
Request declined: Daily token budget exhausted. Quota
                  resets at 00:00 UTC.
```

In benchmark `daily_budget_exhausted`, 15 requests were dispatched. The first consumed 1,331 tokens and retried before budget state was established (1,217.2 ms). Subsequent requests were rejected locally in 3.4–19.2 ms (median: 8.3 ms, p95: 394.6 ms). Waiting users receive an immediate response rather than stalling on network timeouts.

8. What I'm unsure about, what I left out, what I learned

Unsure about: Groq's speculative decoding token metering: if rejected speculative candidates are billed, client-side counters underestimate actual quota burn.

Left out: (1) *Dense Vector DB (Scope Choice):* With 337 chunks, in-memory BM25 executes in < 3 ms with zero network cost; vector DBs add operational fragility. (2) *Unverified Streaming (Scope Choice):* Buffered verification prevents releasing unverified drafts. (3) *Multimodal Audio (Limitation):* Text-only execution.

Learned: Default SDK retry policies are treacherous under load. `AsyncGroq` defaults to internal retries with exponential backoff that mask 429s, causing silent stalling. Setting `max_retries=0` and handling backpressure explicitly is mandatory.

Notes

Vaani feedback: I liked the full-duplex voice architecture (`https://www.vaani.dev/`); I would document barge-in cancellation semantics and expose VAD sensitivity.

Authorship: The system design choices and all measurements reported here are mine; the full prompt/trace history is archived in `transcripts/conversation_history.jsonl`.